Blatt 2

Kryptologie

Luis Staudt

Geschwindigkeitsvergleich Block- vs. Stream-Cipher

Aufgabenstellung

Es sollen fünf Verschlüsselungsverfahren hinsichtlich ihrer Geschwindigkeit verglichen werden:

- **ChaCha20** – Stream-Cipher (256-Bit Schlüssel, 96-Bit Nonce)
- **AES-256-CBC** – Block-Cipher (256-Bit Schlüssel, 128-Bit IV, CBC-Mode)
- **ARC4** – Stream-Cipher (256-Bit Schlüssel)
- **Blowfish-CBC** – Block-Cipher (128-Bit Schlüssel, 64-Bit IV, CBC-Mode)
- **Serpent-CBC** – Block-Cipher (256-Bit Schlüssel, 128-Bit IV, CBC-Mode)

Dazu wird ein zufälliger Plaintext der Größe 2^n Bytes ($n = 10, \ldots, 26$, d. h. 1 KB bis 64 MB) mit jedem Verfahren verschlüsselt und die Verschlüsselungsdauer gemessen. Durch Mittelung über mehrere Wiederholungen wird die Streuung reduziert.

Vorgehensweise

Die Implementierung erfolgt in Java unter Verwendung der Bouncy-Castle-Bibliothek (Version 1.80). Für jeden Plaintext und jedes Verfahren werden zunächst 3 Warmup-Durchläufe durchgeführt, damit der JIT-Compiler die relevanten Code-Pfade optimieren kann. Anschließend wird die Verschlüsselungsdauer über 10 Messungen gemittelt.

Für jeden Durchlauf wird ein zufälliger 256-Bit Schlüssel sowie eine zufällige Nonce bzw. ein zufälliges IV erzeugt. Bei den Block-Ciphern (AES, Blowfish, Serpent) kommt der CBC-Mode mit PKCS7-Padding zum Einsatz. ChaCha20 und ARC4 werden als Stream-Cipher direkt ohne Padding verwendet.

```java
static byte[] encryptBlockCipher(BlockCipher engine, byte[] key,
        int ivSize, byte[] plaintext) throws Exception {
    byte[] iv = new byte[ivSize];
    RANDOM.nextBytes(iv);
    PaddedBufferedBlockCipher cipher = new PaddedBufferedBlockCipher(
        CBCBlockCipher.newInstance(engine), new PKCS7Padding());
    CipherParameters params = new ParametersWithIV(
        new KeyParameter(key), iv);
    cipher.init(true, params);
    byte[] output = new byte[cipher.getOutputSize(plaintext.length)];
    int len = cipher.processBytes(plaintext, 0,
        plaintext.length, output, 0);
    cipher.doFinal(output, len);
    return output;
}
```

Code Listing 1: Verschlüsselung mit einem Block-Cipher (CBC-Mode)

```java
static byte[] encryptChaCha20(byte[] key, byte[] plaintext)
        throws Exception {
    byte[] nonce = new byte[12];
    RANDOM.nextBytes(nonce);
    StreamCipher cipher = new ChaCha7539Engine();
    cipher.init(true,
        new ParametersWithIV(new KeyParameter(key), nonce));
    byte[] output = new byte[plaintext.length];
    cipher.processBytes(plaintext, 0,
        plaintext.length, output, 0);
    return output;
}
```

Code Listing 2: Verschlüsselung mit ChaCha20

Ergebnisse

Einzelne Laufzeiten

Die folgende Tabelle zeigt die mittlere Verschlüsselungsdauer und den Durchsatz für eine Plaintextgröße von $2^{20} = 1$ MB:

Verfahren	Dauer (ms)	Durchsatz (MB/s)
ChaCha20	2,05	488,37
AES-256-CBC	4,12	242,96
ARC4	2,07	483,29
Blowfish-CBC	7,07	141,53
Serpent-CBC	8,80	113,60

Tabelle 1: Verschlüsselungsdauer und Durchsatz bei 1 MB Plaintext

Vergleichsdiagramm

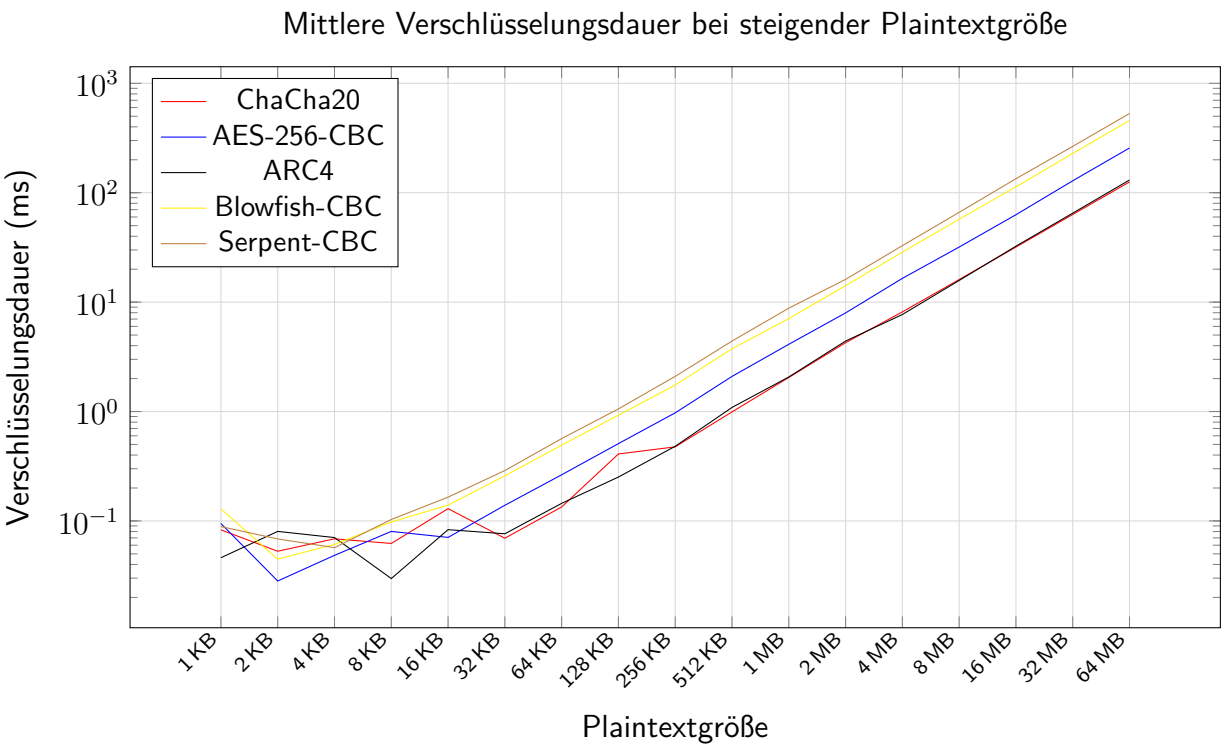

Abbildung 1: Logarithmische Darstellung der Verschlüsselungsdauer aller Verfahren

Einzeldiagramme

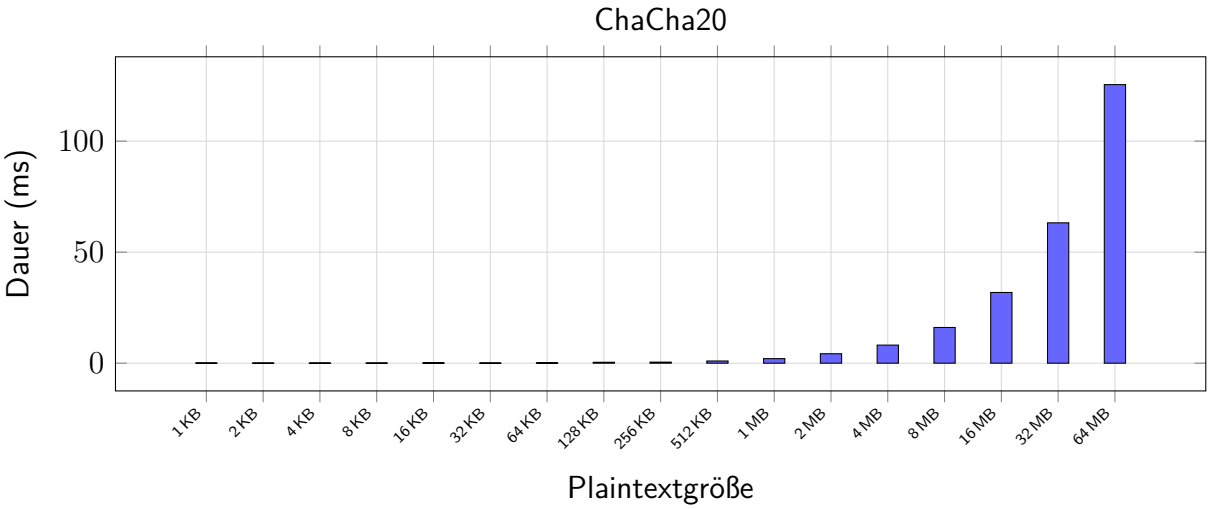

Abbildung 2: Verschlüsselungsdauer von ChaCha20

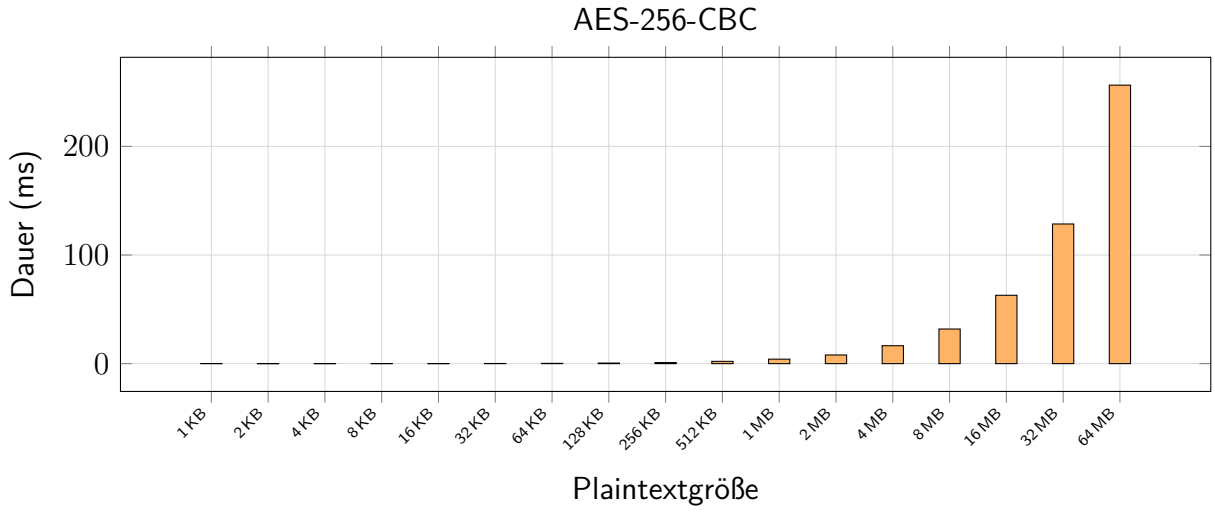

Abbildung 3: Verschlüsselungsdauer von AES-256-CBC

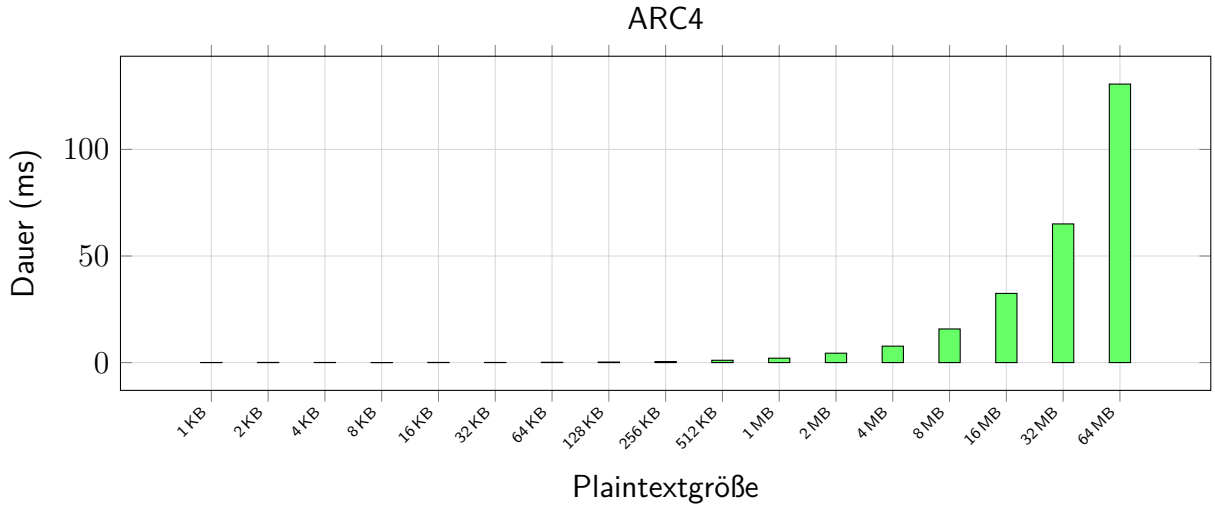

Abbildung 4: Verschlüsselungsdauer von ARC4

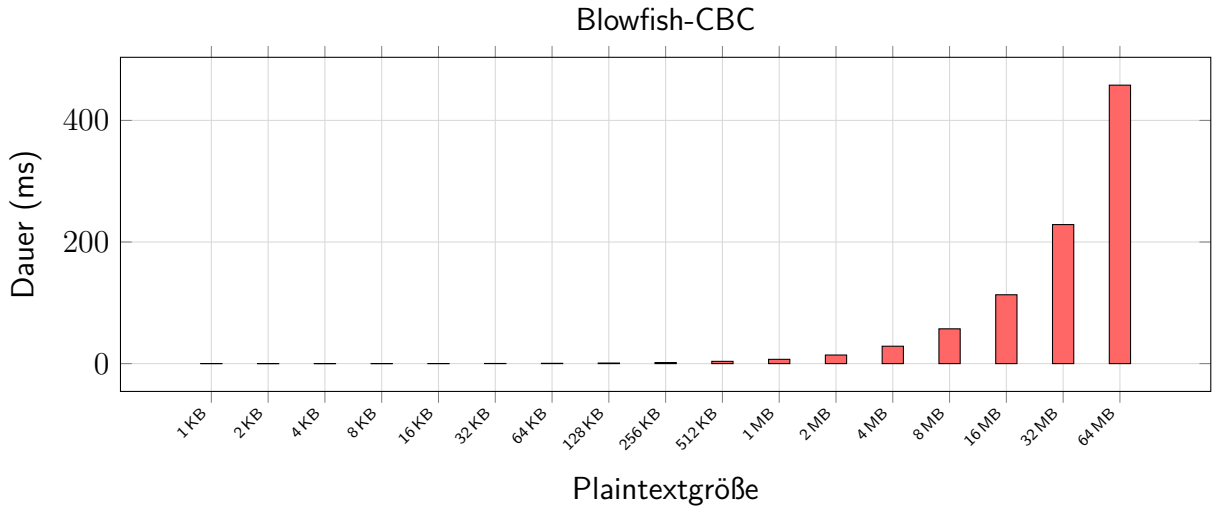

Abbildung 5: Verschlüsselungsdauer von Blowfish-CBC

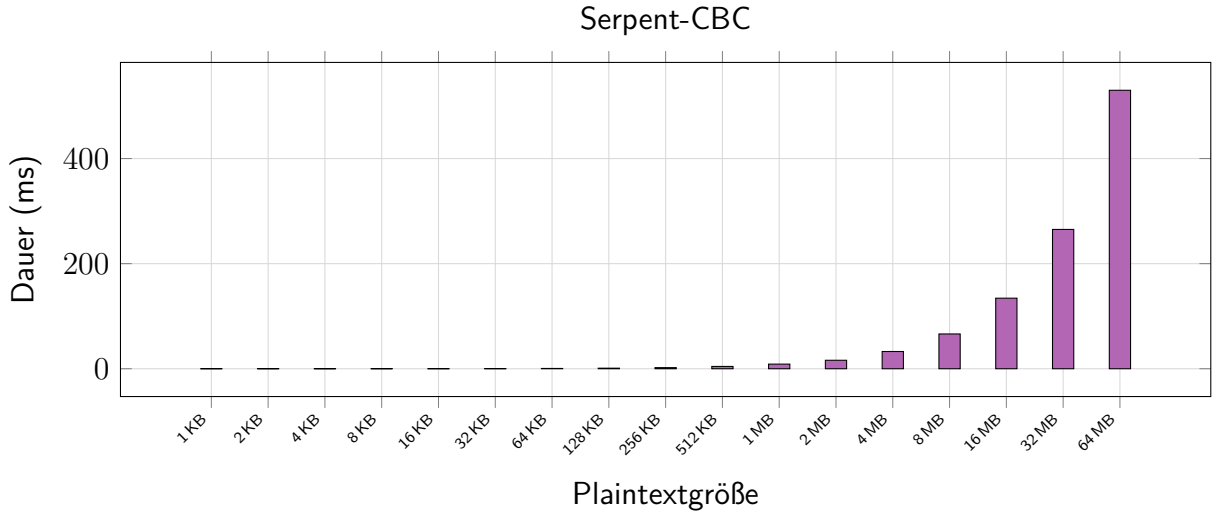

Abbildung 6: Verschlüsselungsdauer von Serpent-CBC

Durchsatzvergleich

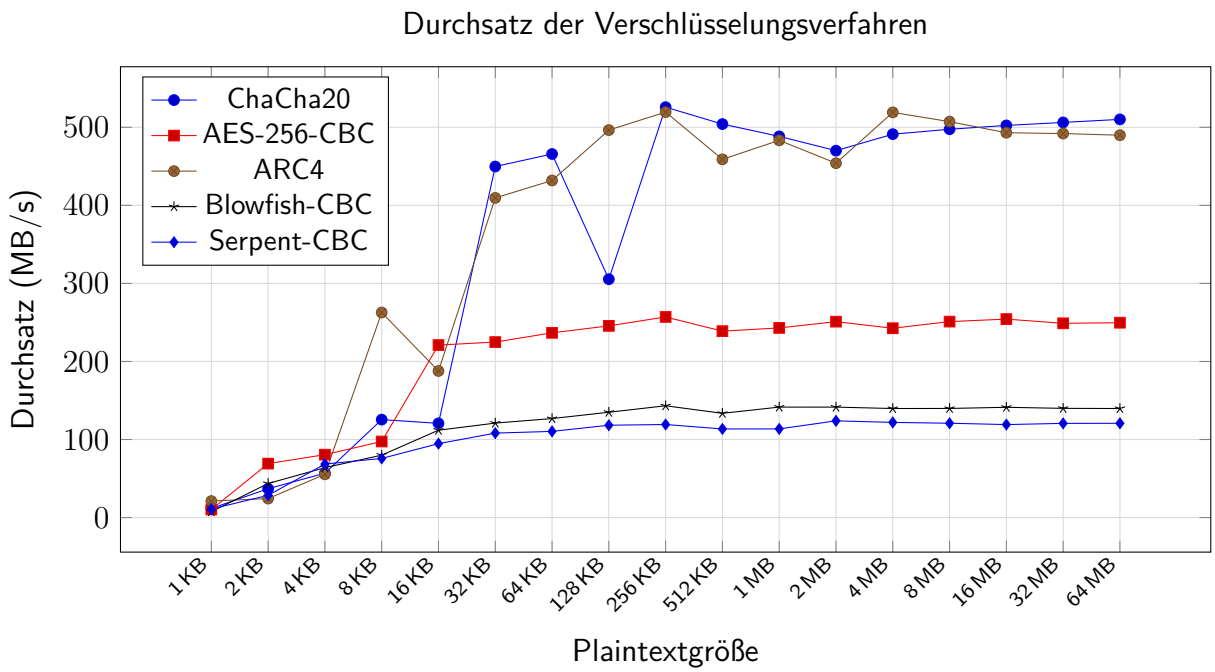

Abbildung 7: Durchsatz in MB/s bei steigender Plaintextgröße

Beobachtungen

Einzelne Verfahren

- **ChaCha20:** Mit einem stabilen Durchsatz von ca. 490–510 MB/s gehört ChaCha20 zu den schnellsten Verfahren. Die Laufzeit skaliert nahezu linear mit der Plaintextgröße. Bei kleinen Datenmengen (< 16 KB) dominiert der Overhead der Initialisierung.
- **AES-256-CBC:** AES erreicht einen Durchsatz von ca. 240–255 MB/s. Im CBC-Mode ist AES nicht parallelisierbar (bei der Verschlüsselung), da jeder Block vom vorherigen Ciphertext-Block abhängt. Trotzdem ist die Leistung dank optimierter Bouncy-Castle-Implementierung solide.
- **ARC4:** ARC4 zeigt mit ca. 480–520 MB/s ähnliche Geschwindigkeit wie ChaCha20. Als einfacher Stream-Cipher hat ARC4 sehr wenig Overhead pro Byte, was die hohe Geschwindigkeit erklärt. ARC4 gilt allerdings als kryptographisch unsicher.
- **Blowfish-CBC:** Blowfish erreicht ca. 135–142 MB/s. Die geringere Geschwindigkeit gegenüber AES erklärt sich durch die kleinere Blockgröße von 64 Bit (doppelt so viele Blöcke pro Datenmenge) und die aufwendige Feistel-Struktur.

- **Serpent-CBC:** Serpent ist mit ca. 115–125 MB/s das langsamste Verfahren. Serpent verwendet 32 Runden (im Vergleich zu 14 Runden bei AES-256), was die höhere Sicherheitsmarge, aber auch die geringere Geschwindigkeit erklärt.

Vergleich

Die Stream-Cipher (ChaCha20, ARC4) sind deutlich schneller als die Block-Cipher im CBC-Mode. Dies liegt daran, dass Stream-Cipher den Plaintext byteweise mit dem Keystrom XOR-verknüpfen, ohne auf Block-Padding oder Block-Verkettung angewiesen zu sein.

Im Vergleich zur Aufgabenstellung mit Python-Bibliotheken fällt auf, dass die Java/Bouncy-Castle-Implementierung durchgehend hohe Durchsatzwerte liefert. Insbesondere Serpent profitiert enorm: Während die Python-Implementierung (pyserpent) bei 1 MB bereits 13,8 Sekunden benötigte (0,07 MB/s), erreicht die Java-Implementierung ca. 113 MB/s – ein Faktor von über 1600. Dies zeigt den erheblichen Einfluss der Laufzeitumgebung und des JIT-Compilers auf die Benchmark-Ergebnisse.

Alle Verfahren zeigen ab ca. 2^{15} Bytes (32 KB) ein lineares Skalierungsverhalten, wobei der Durchsatz stabil bleibt. Bei kleineren Datenmengen dominiert der Initialisierungsoverhead, was zu unregelmäßigen Messwerten führt.

Bewertung

Der Geschwindigkeitsvergleich zeigt eine klare Hierarchie:

1. ChaCha20 und ARC4 (∼500 MB/s) – Stream-Cipher mit minimalem Overhead
2. AES-256-CBC (∼250 MB/s) – vielseitiger Block-Cipher, Industriestandard
3. Blowfish-CBC (∼140 MB/s) – älterer Block-Cipher mit kleiner Blockgröße
4. Serpent-CBC (∼120 MB/s) – hohe Sicherheitsmarge, aber geringerer Durchsatz

Für die Praxis ist zu beachten, dass ARC4 trotz seiner Geschwindigkeit aufgrund bekannter Schwachstellen nicht mehr verwendet werden sollte. ChaCha20 bietet die beste Kombination aus Geschwindigkeit und Sicherheit und wird daher zunehmend in modernen Protokollen wie TLS 1.3 eingesetzt. AES bleibt aufgrund von Hardware-Beschleunigung (AES-NI) in der Praxis auf vielen Plattformen das schnellste Verfahren, wobei die hier verwendete reine Software-Implementierung von Bouncy Castle keine Hardware-Beschleunigung nutzt.